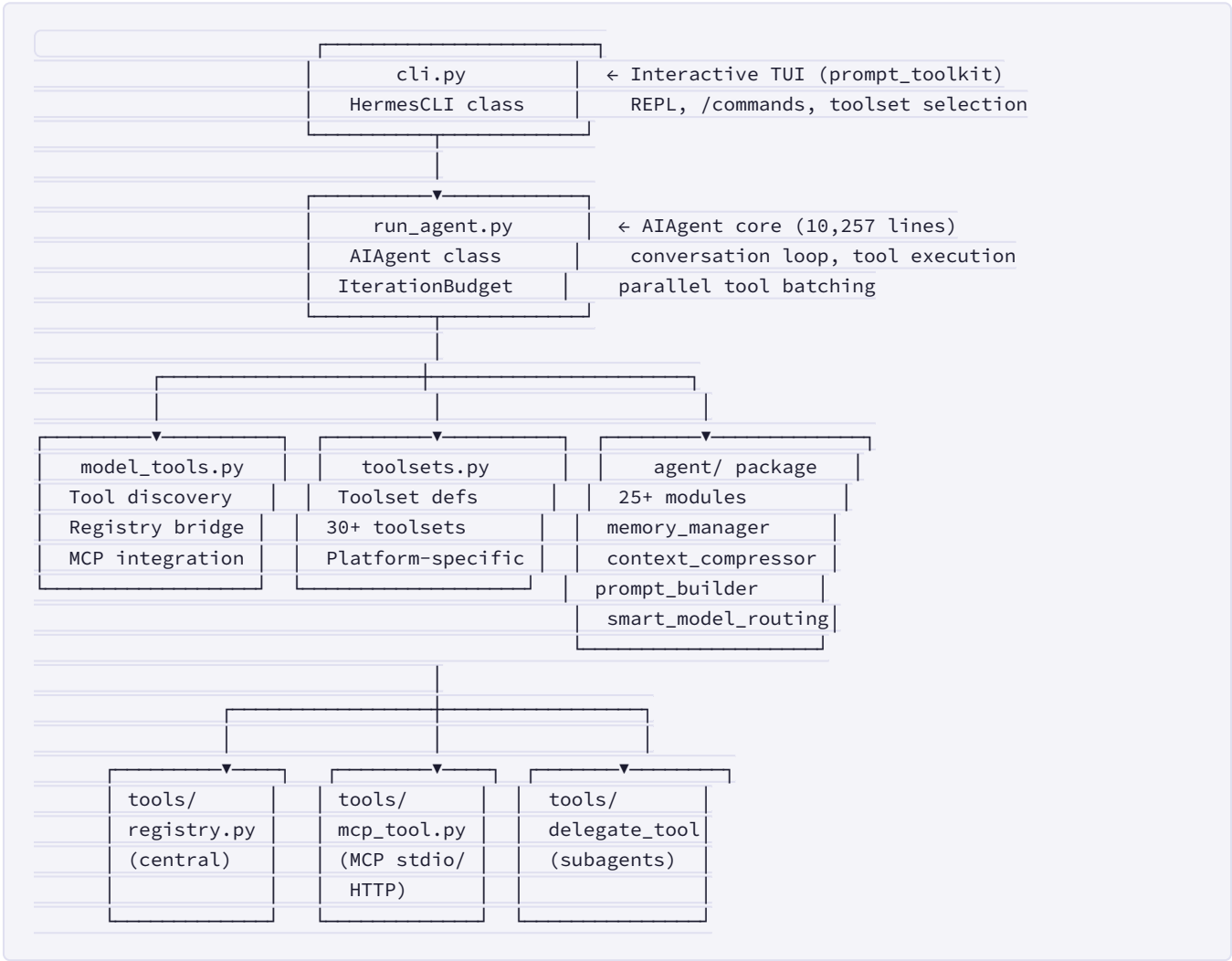


Hermes Agent System Evaluation Report

Date: June 2, 2026
System: Hermes Agent (Nous Research)
Path: /home/opc/.hermes/hermes-agent

1. System Architecture

HERMES AGENT ARCHITECTURE |



| SUPPORTING
SYSTEMS | | cron/ |
gateway/ | acp_adapter/ | scheduler.py | platforms/ | server.py | jobs.py | 15 platforms | session.py
| tick loop | base.py | ACP stdio ↔ IDE |

Core Module Responsibilities

Module	Lines	Purpose
<code>run_agent.py</code>	~10,257	Main agent loop, tool batching, message history, API calls (OpenAI/Anthropic/multi-provider)
<code>model_tools.py</code>	~578	Tool discovery via import, registry bridge, async event loop management, tool call dispatcher
<code>toolsets.py</code>	~667	Toolset definitions (30+ toolsets), recursive toolset resolution, platform-specific bundles
<code>cli.py</code>	~9,381	Interactive terminal UI (prompt_toolkit), config loading, personality system, 12 personas
agent/ package	25+ modules	Memory, context compression, prompt building, smart routing, trajectory, error handling

2. Tool Ecosystem (tools/ directory)

2.1 Registered Tools (via tools/registry.py + self-registration)

Total: 60+ tools across these categories:

File Operations (6 tools)

Tool	Description
<code>read_file</code>	Read file with line numbers, pagination, offset/limit
<code>write_file</code>	Write content (overwrites), creates parent dirs
<code>patch</code>	Fuzzy find-and-replace, 9 matching strategies
<code>search_files</code>	Regex grep + glob glob, count/content/files output
<code>file_operations</code>	Cross-platform file ops (copy, move, mkdir, symlink)
<code>terminal</code>	Command execution (local/ssh/docker/singularity/modal/daytona)

Web & Research (3 tools)

Tool	Description
<code>web_search</code>	Fetch search results (DuckDuckGo/Google Serper)
<code>web_extract</code>	Scrape + summarize web pages (auxiliary LLM)
<code>browser_navigate</code>	CDP-based browser automation (Playwright)

Vision & Media (4 tools)

Tool	Description
<code>vision_analyze</code>	Image analysis (GPT-4o/Vision/local)
<code>image_generate</code>	AI image gen (DALL-E 3, Imagen, Fireworks)
<code>text_to_speech</code>	TTS (Edge TTS/ElevenLabs/OpenAI)
<code>neutts_synth</code>	Neutral TTS synthesis

Skills System (3 tools)

Tool	Description
<code>skills_list</code>	List all available skills
<code>skill_view</code>	Read a skill's prompt/instructions
<code>skill_manage</code>	Create/edit/delete skills

Memory & Session (3 tools)

Tool	Description
<code>memory</code>	Persistent MEMORY.md + USER.md with threat scanning
<code>session_search</code>	FTS5 SQLite search past sessions, summarize via auxiliary LLM
<code>todo</code>	Task planning store (in-memory, per-session)

Delegation & Code (3 tools)

Tool	Description
<code>delegate_task</code>	Spawn subagents, isolated context, max_depth=2, concurrent children
<code>execute_code</code>	Run Python scripts in sandbox with allowed tools list
<code>process</code>	Process management (list, kill signal)

Browser Automation (10 tools)

Tool	Description
<code>browser_navigate</code>	Navigate to URL, handle iframes/tabs
<code>browser_snapshot</code>	Full-page screenshot
<code>browser_click</code>	DOM click with selector
<code>browser_type</code>	Keyboard input
<code>browser_scroll</code>	Page scroll
<code>browser_back/press</code>	Navigation
<code>browser_get_images</code>	Extract image URLs
<code>browser_vision</code>	Visual grounding
<code>browser_console</code>	Console log extraction

Messaging & Platform (2 tools)

Tool	Description
<code>send_message</code>	Cross-platform (Telegram/Discord/Slack/SMS/Email/etc.)
<code>homeassistant_tool</code>	HA entity listing, state, service calls

Scheduler (1 tool)

Tool	Description
<code>cronjob</code>	Create/list/update/pause/resume/trigger/delete/schedule jobs

Other (6 tools)

Tool	Description
<code>mixture_of_agents</code>	Distributed reasoning pipeline
<code>rl_training_tool</code>	RL training on Tinker-Atropos
<code>harness_tools</code>	20+ Harness system tools (workflow, eval, telemetry, sandbox, cost, marketplace, replay)
<code>clarify</code>	User clarifying questions UI
<code>mcp_tool</code>	MCP server tool discovery/forwarding
<code>transcription_tools</code>	Audio transcription

2.2 Tool Toolsets

Toolsets group tools for specific use cases:

Toolset	Tools	Purpose
<code>hermes-cli</code>	~50 tools	Full CLI experience
<code>hermes-acp</code>	~35 tools	Editor integration (no messaging/audio)
<code>hermes-telegram</code>	~50 tools	Telegram bot
<code>hermes-discord</code>	~50 tools	Discord bot
<code>hermes-gateway</code>	union of all	Multi-platform gateway bundling
<code>hermes-api-server</code>	~40 tools	OpenAI-compatible HTTP API
<code>safe</code>	web+vision+image	No terminal access
<code>delegation</code>	delegate_task only	Subagent spawning
<code>harness</code>	20+ harness tools	Workflow engine + eval + telemetry
<code>homeassistant</code>	HA tools only	Smart home control

3. Skill System

3.1 User Skills (~/.hermes/skills/)

36 skill categories installed in user home:

Category	Example Skills
<code>apple</code>	macOS/iOS integration
<code>autonomous-ai-agents</code>	Agent autonomy patterns
<code>central</code>	Symlinks to shared skills
<code>creative</code>	ascii-art, creative-ideation, excalidraw, songwriting
<code>data-science</code>	jupyter-live-kernel, hk-stocks-yahoo-finance, mingpao scraper
<code>devops</code>	cloudflared-quick-tunnel, modal-sandbox-ssh, webhook-subscriptions
<code>github</code>	codebase-inspection, github-auth, github-code-review, github-pr-workflow
<code>media</code>	gif-search, heartmula, youtube-content, youtube-subtitle
<code>mcp</code>	MCP server skills
<code>mlops</code>	accelerate, chroma, faiss, pytorch-lightning, qdrant, pinecone
<code>productivity</code>	Various productivity skills
<code>research</code>	Research-oriented skills
<code>security</code>	Security-focused skills
<code>smart-home</code>	Home automation
<code>social-media</code>	Social platform integrations

3.2 Optional Skills (hermes-agent/optional-skills/)

15 skill categories packaged with the agent:

Category	Sub-skills
<code>autonomous-ai-agents</code>	blackbox, honcho
<code>blockchain</code>	base, solana
<code>communication</code>	one-three-one-rule
<code>creative</code>	blender-mcp, meme-generation
<code>devops</code>	cli, docker-management
<code>mlops</code>	accelerate, chroma, faiss, flash-attention, huggingface-tokenizers, instructor, lambda-labs, llava, nemo-curator, pinecone, pytorch-lightning, qdrant
<code>security</code>	(5 skills)

Skill format: Markdown files with instructions, examples, and domain knowledge that get injected into system prompts.

4. Gateway Messaging Platforms

15 messaging platforms supported in `gateway/platforms/`:

Platform	File	Protocol
Telegram	telegram.py + telegram_network.py	Bot API
Discord	discord.py	Bot API (slash commands)
WhatsApp	whatsapp.py	WhatsApp Business API
Slack	slack.py	WebSocket + HTTP
Signal	signal.py	Signal Messenger
Matrix	matrix.py	Matrix E2EE
Mattermost	mattermost.py	Self-hosted
Email	email.py	IMAP/SMTP
SMS	sms.py	Twilio
DingTalk	dingtalk.py	Alibaba DingTalk
Feishu/Lark	feishu.py	ByteDance Feishu
WeCom	wecom.py	Enterprise WeChat
Weixin	weixin.py	WeChat iLink
BlueBubbles	bluebubbles.py	Apple iMessage
HomeAssistant	homeassistant.py	HA WebSocket REST
Webhook	webhook.py	HTTP inbound
API Server	api_server.py	OpenAI-compatible REST

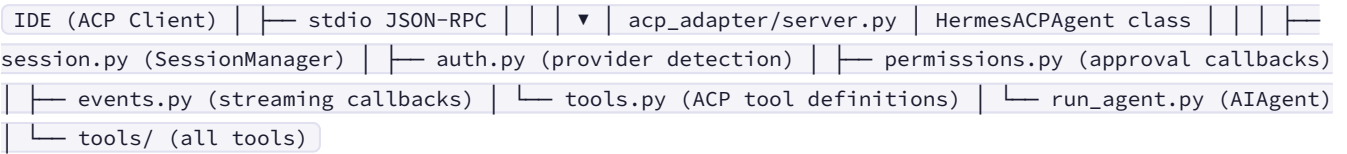
All platforms inherit from `BasePlatformAdapter` with common patterns.

5. ACP Adapter (IDE Integrations)

5.1 Supported IDEs

IDE	Method	Protocol
VS Code	ACP Client extension (<code>anysphere.acp-client</code>)	ACP stdio
Zed	Built-in <code>agent_servers</code> config	ACP stdio
JetBrains (Intellij, PyCharm, etc.)	ACP plugin	ACP stdio

5.2 ACP Architecture



5.3 ACP Toolset

Uses `hermes-acp` toolset: ~35 tools (no `send_message`, `clarify`, `cronjob`, `text_to_speech`)

6. Configuration System

6.1 Config Locations

Config File	Purpose
<code>~/.hermes/config.yaml</code>	User-level (primary)
<code>./cli-config.yaml</code>	Project-level fallback
<code>~/.hermes/.env</code>	Environment variables / API keys
<code>~/.hermes/skills/</code>	Per-skill configuration

6.2 Config YAML Structure

```yaml

Model

model: default: "anthropic/claude-opus-4" # or provider: "openrouter" base\_url: "https://openrouter.ai/v1" # OpenAI-compatible endpoint provider: "auto" # auto/openrouter/nous/anthropic/gemini/etc.

Terminal backend

terminal: backend: "local" # local/ssh/docker/singularity/modal/daytona cwd: "." timeout: 180 lifetime\_seconds: 300 docker\_image: "nikolaik/python-nodejs:python3.11-nodejs20"

Browser

browser: inactivity\_timeout: 120 record\_sessions: false

Compression

compression: enabled: true threshold: 0.50 # compress at 50% context summary\_model: "" # empty = use main model

## Smart routing

smart\_model\_routing: enabled: false max\_simple\_chars: 160 max\_simple\_words: 28 cheap\_model: {}

## Agent

agent: max\_turns: 90 personalities: # 13 pre-built personas kawaii: "You are a kawaii assistant! Use cute expressions..." catgirl: "You are Neko-chan, an anime catgirl..."

## Display / Skins

display: compact: false show\_reasoning: false streaming: true skin: "default" # yaml skin file name

## Clarify

clarify: timeout: 120

## Code execution

code\_execution: timeout: 300 max\_tool\_calls: 50

## Delegation

delegation: max\_iterations: 45 default\_toolsets: ["terminal", "file", "web"] ``

### 6.3 Skins System

Located in `docs/skins/example-skin.yaml` : - Customizable colors (hex values for Rich markup) - Spinner faces and thinking verbs - Branding text (agent name, prompts, goodbye) - Tool output prefix character

Activate with `/skin <name>` or `display.skin: <name>` in config.

## 7. Cron/Scheduler System

### 7.1 Components

| File                           | Purpose                                                         |
|--------------------------------|-----------------------------------------------------------------|
| <code>cron/scheduler.py</code> | <code>tick()</code> — check due jobs, run them, deliver results |
| <code>cron/jobs.py</code>      | CRUD operations, schedule parsing                               |

### 7.2 Scheduler Features

- **File-based locking** ( `~/hermes/cron/.tick.lock` ) prevents concurrent ticks
- **Pre-run scripts** in `~/hermes/scripts/` with timeout control
- **Threat scanning** on cron prompts (prompt injection, exfil patterns)
- **Auto-delivery** to origin platform or explicitly configured channels
- **Supports all 15 messaging platforms** for delivery
- **Parallel execution** of independent due jobs
- **Output wrapping** with configurable header/footer
- **Media file forwarding** via adapter

### 7.3 Cron Tool ( `cronjob` )

Single compressed tool with these actions: - `create` / `list` / `update` / `pause` / `resume` / `remove` / `trigger` / `skip` - Schedule: cron expressions (5-field) - Repeat: `times` (N) or `forever` - Skills: inject skill context before job prompt - Script: pre-run data collection script - Delivery: `local` / `origin` / `platform:chat_id` - Visibility: `public` (all chalk IDs) or `private` (per-chat-ID isolation)

## 8. MCP Integrations

### 8.1 Architecture

| Feature           | Implementation                                            |
|-------------------|-----------------------------------------------------------|
| Transport         | Stdio (command + args) + HTTP/StreamableHTTP (url)        |
| Protocol          | <code>mcp</code> Python package (optional)                |
| Background loop   | Dedicated daemon thread ( <code>_mcp_loop</code> )        |
| Thread safety     | <code>_lock</code> protected, per-thread persistent loops |
| Dynamic discovery | <code>notifications/tools/list_changed</code> support     |
| Sampling          | MCP servers can request LLM completions                   |
| Reconnection      | Exponential backoff, up to 5 retries                      |

### 8.2 Configuration (config.yaml)

```
yaml mcp_servers: filesystem: command: "npx" args: ["-y", "@modelcontextprotocol/server-filesystem", "/tmp"] env: {} timeout: 120 connect_timeout: 60 github: command: "npx" args: ["-y", "@modelcontextprotocol/server-github"] env: GITHUB_PERSONAL_ACCESS_TOKEN: "ghp_..." remote_api: url:
```

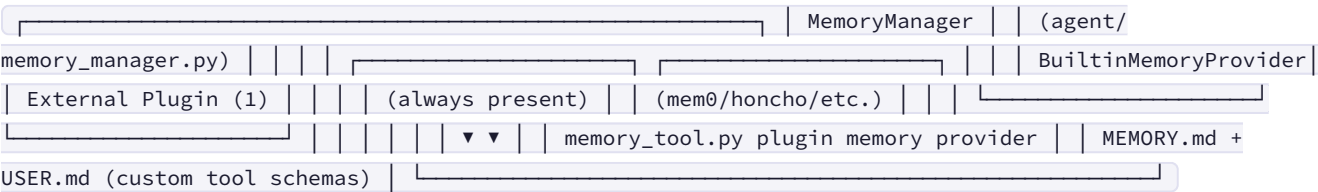
```
"https://my-mcp-server.example.com/mcp" headers: Authorization: "Bearer sk-..." timeout: 180 analysis:
command: "npx" args: ["-y", "analysis-server"] sampling: enabled: true model: "gemini-3-flash"
max_tokens_cap: 4096 max_rpm: 10
```

8.3 Security

- Safe env filtering (PATH, HOME, USER, LC\_ALL, TERM, SHELL, TMPDIR + XDG\_\*)
- Credential stripping from error messages (ghp\_, sk-, Bearer, token=, API\_KEY=, etc.)
- 15 fixed credential patterns redacted

9. Memory System

9.1 Built-in Memory ( memory tool + MemoryManager )



**Memory architecture:** - `MemoryStore` class with bounded entries - Delimiter: `§` (section sign) - Entry operations: `add`, `replace`, `remove`, `read` - Threat scanning: prompt injection + exfil + invisible unicode - Frozen snapshot pattern: system prompt snapshot, mid-session writes don't invalidate it

**Two stores:** | Store | Purpose | File | |-----|-----|-----| | `MEMORY.md` | Agent's personal notes, environment facts, project conventions | `~/.hermes/memories/` | | `USER.md` | User preferences, communication style, workflow habits | `~/.hermes/memories/` |

9.2 Session Search ( session\_search tool)

- SQLite FTS5 full-text search across all past sessions
- Groups results by session, takes top 3
- Truncates to ~100k chars centered on match
- Summarizes via auxiliary LLM (Gemini Flash by default)
- Returns: date, source, summary, turn count

**Session data flow:** `run_agent.py` — `_session_db` (optional SQLite, provided by CLI/gateway) — `create_session(session_id, source, model, ...)` — `append_message(session_id, role, content, tool_calls, ...)` — `session_search(query)` → FTS5 → summarize → results

9.3 Memory Plugin Ecosystem

6 external memory providers supported via `plugins/memory/`:

| Provider    | Package           | Description                                                   |
|-------------|-------------------|---------------------------------------------------------------|
| mem0        | mem0ai            | Server-side LLM fact extraction + semantic search + reranking |
| honcho      | honcho-dev/honcho | Per-user/proj memory with CLI web interface                   |
| supermemory | supermemory       | Web memory/tweet storage                                      |
| retaindb    | (custom)          | Database-backed retention                                     |
| openviking  | (custom)          | Viking memory                                                 |
| holographic | (custom)          | Holographic memory                                            |
| byterover   | (custom)          | ByteRover memory                                              |
| hindsight   | (custom)          | Hindsight memory                                              |

**Constraint:** Only ONE external (non-builtin) memory provider allowed at a time.

## 10. Subagent Delegation System

### 10.1 Architecture ( `delegate_task` tool)

```

Parent AIAgent |— delegate_task(goal, context, toolsets) | |— Spawn N child AIAgent instances
(ThreadPoolExecutor) | |— Each child: isolated context, own task_id, restricted toolsets | |— Default
toolsets: terminal, file, web | |— Max iterations per child: 50 (configurable) | |— Returns: JSON
summary (what/done/modified/issues) | |— Batch mode supported (parallel execution, max 3 concurrent)

```

### 10.2 Delegation Configuration

```

yaml delegation: max_iterations: 45 # Per child agent (default 50 in tool) default_toolsets:
["terminal", "file", "web"] model: "" # Empty = inherit parent model provider: "" # Empty = inherit
parent provider base_url: "" # Direct endpoint for subagents max_concurrent_children: 3 # Default 3

```

### 10.3 Blocked Tools (Never Given to Children)

```

python DELEGATE_BLOCKED_TOOLS = frozenset(["delegate_task", # no recursive delegation "clarify", # no
user interaction "memory", # no shared memory writes "send_message", # no cross-platform side effects
"execute_code", # children should reason step-by-step])

```

### 10.4 Progress Callback

CLI mode: prints tree-view lines with tool emoji Gateway mode: batches tool names, relays to parent's progress callback

## 11. System Strengths

### 11.1 Architecture Quality

- **Modular registry pattern:** Each tool self-registers, easy to add new tools
- **Clean separation:** `model_tools.py` bridges registry → API, `toolsets.py` organizes groupings
- **Async-first design:** Persistent event loops per thread prevent "event loop is closed" errors
- **Hierarchical toolsets:** Inclusions support diamond dependencies without cycles

- **Parallel tool batching:** 8 worker threads, path-scoped conflict detection for file ops

## 11.2 Multi-Provider Flexibility

- 20+ inference providers: OpenRouter (with routing preferences), Nous Portal (OAuth or API key), Anthropic (native), OpenAI Codex, GitHub Copilot, Google Gemini, Z.ai, Kimi, MiniMax, HuggingFace, KiloCode, Vercel AI Gateway, local (LM Studio/Ollama/vLLM/llama.cpp)
- Smart model routing: auto-switch to cheap model for simple queries
- Credential refresh: Nous, Anthropic, Codex token refresh
- Prompt caching: native Anthropic or Claude-via-OpenRouter

## 11.3 Comprehensive Tool Suite

- 60+ tools covering file ops, web, vision, media, skills, memory, delegation, browser automation, home automation, coding, scheduling, messaging
- 15 messaging platforms, 6 memory providers, RL training, Harness ecosystem
- Browser automation via CDP with Playwright + CamoFox + BrowserBase + Firecrawl providers

## 11.4 Security Hardening

- Memory threat scanning (prompt injection, exfil, invisible unicode)
- Cron prompt threat scanning
- Safe env filtering for MCP stdio subprocesses
- Credential stripping from error messages
- Path traversal guards in cron scripts
- Destructive command detection in terminal

## 11.5 Extreme Customizability

- 13 built-in personas (+ make your own)
- YAML skin system with full color/spinner/branding customization
- 36 user skill categories + 15 optional skill packages
- Configurable in 20+ categories (model, terminal, browser, compression, routing, agent, display, clarify, code\_execution, delegation, auxiliary models, etc.)

## 11.6 ACP-Native Editor Integration

- VS Code, Zed, JetBrains all supported via ACP stdio
  - Session management, resume, fork
  - Approval flow for destructive operations
  - Streaming tool progress callbacks
  - MCP dynamic tool discovery support
-

## 12. Current Gaps & Limitations

### 12.1 Architecture Complexity

| Gap                                         | Impact                                                       |
|---------------------------------------------|--------------------------------------------------------------|
| <code>run_agent.py</code> is 10,257 lines   | Single large file; should be decomposed into smaller modules |
| <code>cli.py</code> is 9,381 lines          | TUI code mixed with config logic; should be modularized      |
| <code>agent/</code> package has 25+ modules | Inconsistent organization (some could be grouped)            |

### 12.2 Missing Capabilities

| Gap                                   | Impact                                                   |
|---------------------------------------|----------------------------------------------------------|
| No native iOS/Android mobile app      | Telegram/WhatsApp/webhook only; no native mobile UX      |
| No video generation tool              | Only image gen (DALL-E, Imagen), not video               |
| No voice input                        | Text only; voice_mode exists but not fully functional    |
| No official database tool             | No SQL query tool; file-based memory only                |
| No document processing                | No PDF/Word/Excel parsing tool                           |
| Limited Kubernetes support            | Docker/singularity/modal, but no k8s operator            |
| No Argo Workflows/Prefect integration | Only cron scheduling; no advanced workflow orchestration |
| Limited monitoring integration        | No Datadog/NewRelic/Grafana agent tools                  |

### 12.3 Security Gaps

| Gap                                                 | Impact                                   |
|-----------------------------------------------------|------------------------------------------|
| <code>sudo_password</code> in plaintext config.yaml | Security risk; should use secret manager |
| No audit logging for cron job runs                  | Compliance gap                           |
| No TLS cert validation for HomeAssistant            | Connection security                      |
| Cron scripts dir not configurable                   | Path traversal risk mitigated but fixed  |

### 12.4 Scalability Gaps

| Gap                                         | Impact                                                                |
|---------------------------------------------|-----------------------------------------------------------------------|
| No multi-user/multi-tenant support          | All sessions share memory; workspace isolation not enforced           |
| No horizontal scaling                       | Single process; no Redis/distributed session store                    |
| ACP concurrent sessions capped at 4 threads | <code>_executor = ThreadPoolExecutor(max_workers=4)</code> bottleneck |
| Session search uses local SQLite FTS5       | No distributed search; limited to single-node                         |

## 12.5 Operational Gaps

| Gap                                     | Impact                                             |
|-----------------------------------------|----------------------------------------------------|
| No official Helm chart or k8s manifests | Docker image exists but no container orchestration |
| No official Prometheus metrics endpoint | No observability integration                       |
| No health check endpoint                | Gateway liveness/readiness not exposed             |
| No structured log output (JSON logging) | Difficult to parse in log aggregation systems      |

## 12.6 Developer Experience

| Gap                           | Impact                                                           |
|-------------------------------|------------------------------------------------------------------|
| No TypeScript SDK             | Only Python                                                      |
| No WebAssembly component      | Can't run in browser                                             |
| Complex .env loading cascade  | Difficult to debug config issues                                 |
| Skills sync mechanism unclear | <code>skills_sync.py</code> exists but unclear how it integrates |
| No visual workflow builder    | Only YAML-based Harness workflows                                |

## 13. Summary Numbers

| Dimension                 | Count                       |
|---------------------------|-----------------------------|
| Python files (main)       | 70+                         |
| Tools registered          | 60+                         |
| Toolsets defined          | 30+                         |
| Messaging platforms       | 15                          |
| Skill categories (user)   | 36                          |
| Optional skill categories | 15                          |
| MCP servers               | unlimited (config)          |
| Memory providers          | 8 (1 builtin + 7 plugin)    |
| Inference providers       | 20+                         |
| ACP IDE integrations      | 3 (VS Code, Zed, JetBrains) |
| Config options (approx)   | 100+                        |
| Personas                  | 13 pre-built + skinnable    |
| Run agent lines           | 10,257                      |
| CLI lines                 | 9,381                       |

*Report generated: June 2, 2026*

*Hermes Agent evaluation for evolution planning*